SORBONNE UNIVERSITÉ
Faculté des Sciences et Ingénierie
Master SESI
Département Informatique

COMPTE RENDU PROJET FPGA

Conception d'une Centrale DCC sur FPGA

Présenté par :
Stéphane Ji
Romaric Chaffray

Encadré par :
Khalil Hachicha

Table des matières

Contexte 3
Architecture Matérielle de la centrale DCC 4
- Générateur de Trames 4
  - Interface du Module (Ports) 4
  - Logique de Décodage et Fonctionnalités 5
  - Structure des Trames DCC 5
  - Validation et Simulation 5
- Registre DCC 6
- Machine à états (MAE) 8
  - Ports de la MAE 8
  - Description des États 8
  - Validation et Simulation 8
- DCC_Bit 1 et DCC_Bit_0 9
- Centrale DCC 9
- TOP_DCC 10

Intégration de la Centrale DCC comme IP Microblaze 11
- Architecture globale du système et interconnexion par Bus AXI 11
- Conception de l'AXI Wrapper et choix des registre d'interface 11
- Gestion des Entrées/Sorties par le Microblaze (GPIO Interrupteurs et Boutons) 12

Architecture Logicielle 13
- Organigramme de constitution des trames DCC 13
  - Configuration Globale des Interrupteurs 13
  - Description des Modes 13
    - Gestion de l'Adresse (`Switch(0)`) 13
    - Sélection du Mode (`Switch(1)` et `Switch(2)`) 13
    - Rôle des Switchs d'Arguments (`Switch(3)` à `Switch(6)`) 13
    - Rôle du Switch Spécial (`Switch(7)`) 14
- Interface Matériel/Logiciel (IP Custom et Application Vitis) 14
  - Architecture de l'IP Custom (Côté Hardware) 14
  - Développement de l'Application (Côté Software sous Vitis) 14
  - Bilan d'avancement : 15

Contexte

Le protocole DCC (Digital Command Control) est un standard de modélisme ferroviaire permettant de contrôler individuellement des locomotives en modulant la tension d'alimentation des voies.

L'objectif de ce projet est de concevoir une « Centrale DCC » embarquée sur un FPGA (carte Basys 3) sous la forme d'un système mixte matériel/logiciel. Le développement s'articule autour de deux approches complémentaires :

L'implémentation matérielle : Elle permet de garantir le respect strict des contraintes temporelles du protocole DCC (de l'ordre de la microseconde) pour la génération des signaux.

L'implémentation logicielle (C / Microblaze) : Le langage C offre une meilleure portabilité et simplifie la gestion de l'interface utilisateur (boutons, interrupteurs) pour constituer les trames de commande. Ces trames sont ensuite transmises à la partie matérielle (transformée en IP) via un bus AXI. Le signal numérique final généré par le FPGA est ensuite amplifié en courant par une carte Booster avant d'être envoyé sur les rails pour être décodé par les trains.

Architecture Matérielle de la centrale DCC

L'architecture matérielle de la centrale DCC est composé de :
- Générateur de Trames
- MAE
- DCC_Bit 1 et 0
- Registre DCC
- Diviseur Horloge
- Compteur Temporisateur

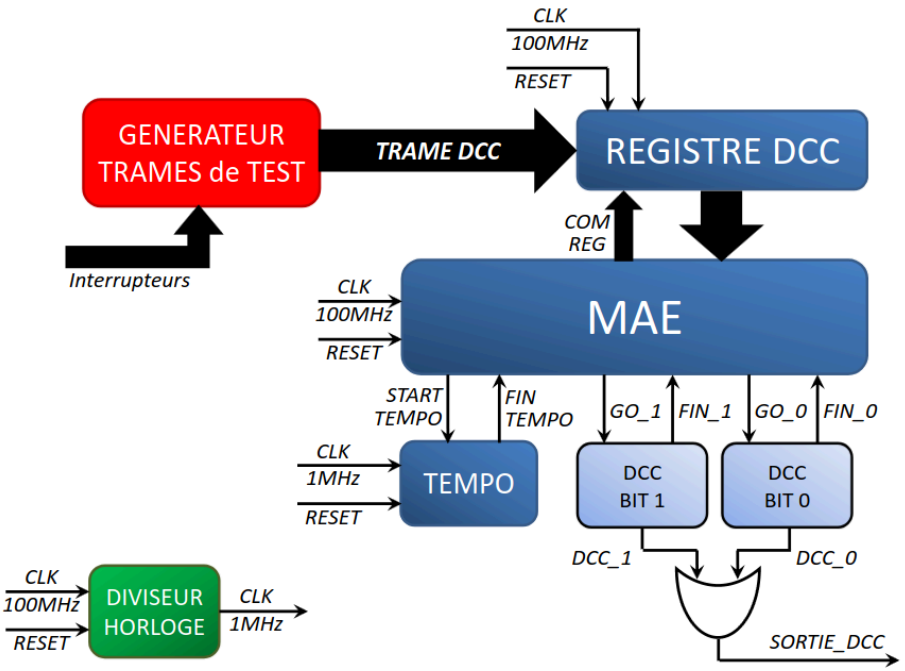


Fig. 1. – Architecture de la Centrale DCC

Les modules `Diviseur Horloge` et `Compteur Temporisateur` étant fournis, notre travail a consisté à concevoir les autres blocs en VHDL. Nous avons également rédigé les fichiers de simulation (test-benchs) pour l'intégralité des composants afin d'en valider le fonctionnement unitaire puis global.

Générateur de Trames

Contrairement à la machine à états, le Générateur de Trames est un module entièrement combinatoire. Son rôle est de traduire instantanément l'état des commutateurs physiques en une trame DCC normalisée de 51 bits.

Interface du Module (Ports)

Le module communique via les signaux suivants :
- **Entrée :** `Interrupteur` (vecteur de 8 bits), représentant l'état des switchs physiques.
- **Sortie :** `Trame_DCC` (vecteur de 51 bits), correspondant à la trame générée.

Logique de Décodage et Fonctionnalités

Pour l'architecture interne, l'accès direct par masquage de bits (par exemple via un test du type `if Interrupteur(7) = '1'`) a été privilégié à une approche arithmétique pour sa simplicité de mise en œuvre.

Chaque interrupteur est ainsi assigné à une commande spécifique du train :
- **Interrupteur(7) :** Marche avant du train
- **Interrupteur(6) :** Marche arrière du train
- **Interrupteur(5) :** Allumage des phares
- **Interrupteur(4) :** Extinction des phares
- **Interrupteur(3) :** Activation du klaxon
- **Interrupteur(2) :** Réamorçage du klaxon
- **Interrupteur(1) :** Annonce SNCF
- **Interrupteur(0) :** Arrêt d'urgence / Configuration par défaut

Si aucun interrupteur n'est activé ou si une configuration non répertoriée apparaît, le système force l'arrêt de sécurité du train.

Structure des Trames DCC

Le module génère deux types de structures selon la complexité de la fonction demandée, tout en respectant la contrainte stricte de 51 bits sur le bus :

1. **Trame Standard (Fonctions simples) :** Utilise un préambule complet de 23 bits, suivi de l'adresse, d'un octet de commande et de l'octet de contrôle (XOR).
2. **Trame Étendue (Fonctions complexes) :** Nécessite un octet de commande supplémentaire. Afin de maintenir la taille globale à 51 bits, le préambule matériel est automatiquement réduit à 14 bits.

Voici un exemple de structure de code pour une trame standard et une trame étendue :

```
-- Exemple : Trame Standard
Trame_DCC <=
  "11111111111111111111111"-- Préambule
(23b)
  & '0'                     -- Start Bit
  & "00000101"              -- Adresse
  & '0'                     -- Start Bit
  & "01100010"    -- Commande (1 octet)
  & '0'                     -- Start Bit
  & "01111010"        -- Contrôle (XOR)
  & '1';                    -- Stop Bit
```

```
-- Exemple : Trame Étendue
Trame_DCC <=
  "11111111111"        -- Préambule réduit
  & '0'                     -- Start Bit
  & "00000101"              -- Adresse
  & '0'                     -- Start Bit
  & "11011110"        -- Commande (Octet 1)
  & '0'                     -- Start Bit
  & "00000001"        -- Commande (Octet 2)
  & '0'                     -- Start Bit
  & "11011010"              -- Contrôle
  & '1';                    -- Stop Bit
```

Validation et Simulation

La validation de ce bloc combinatoire s'appuie sur un banc de test (testbench) sans horloge. Les simulations permettent de vérifier deux aspects critiques :
- La bonne génération de la trame associée à chaque interrupteur.
- La gestion des priorités en cas de conflit (lorsque plusieurs interrupteurs sont activés simultanément).

Les stimuli injectent des valeurs d'exemples à intervalles réguliers (toutes les 20 ns) afin d'observer la réactivité du circuit :

```
-- Séquence de test des interrupteurs
Interrupteur <= "00000000"; wait for 20 ns; -- Arrêt du train
Interrupteur <= "10000000"; wait for 20 ns; -- Marche Avant
Interrupteur <= "10100000"; wait for 20 ns; -- Cas de conflit simultané
```

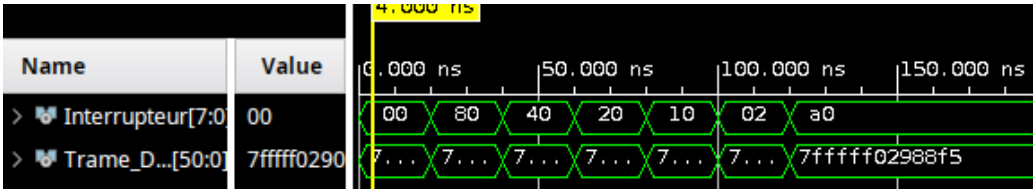


Fig. 2. – Chronogramme de simulation du Générateur de Trames combinatoire

Registre DCC

Ports du Registre DCC

- **Entrée :** `Clk_100`, `Reset`, `TRAME_DCC`, `COM_REG`
- **Sortie :** `BIT_OUT`

Registre interne « registre » = 51 bits

Pour COM_REG = 0b01 => Chargement du TRAME_IN dans registre
Pour COM_REG = 0b10 => Décalage vers la gauche (poids faible vers poids fort)
Pour COM_REG = 0b00 => Attente

Le signal BIT_OUT est relié au registre(50).

Le module Registre DCC possède 3 tâches :

- Chargement de la trame
- Envoi d'un bit de la trame
- Maintien de la trame

Le choix de la tâche dépend de la valeur du signal COM_REG envoyé par le MAE.

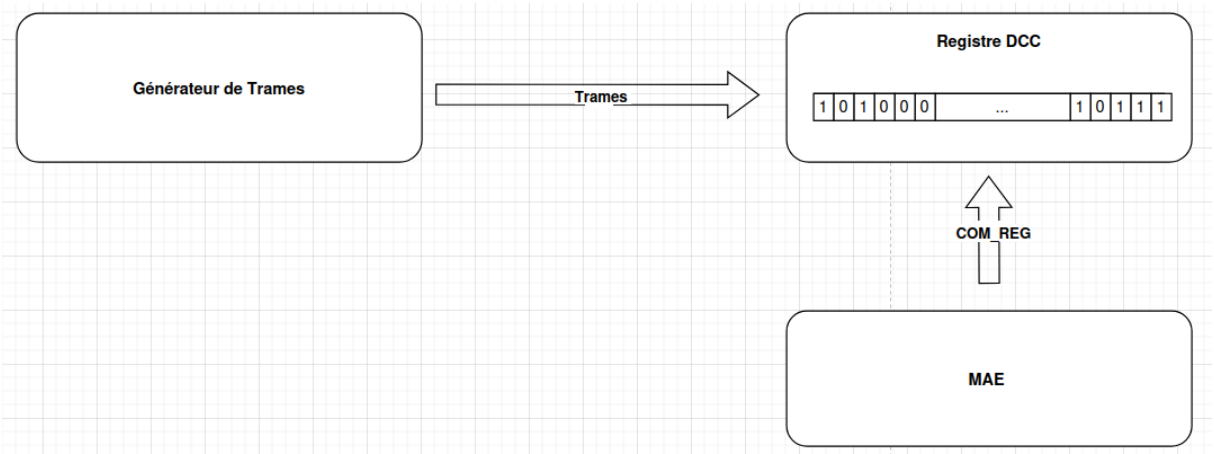


Fig. 3. – Chargement de la trame

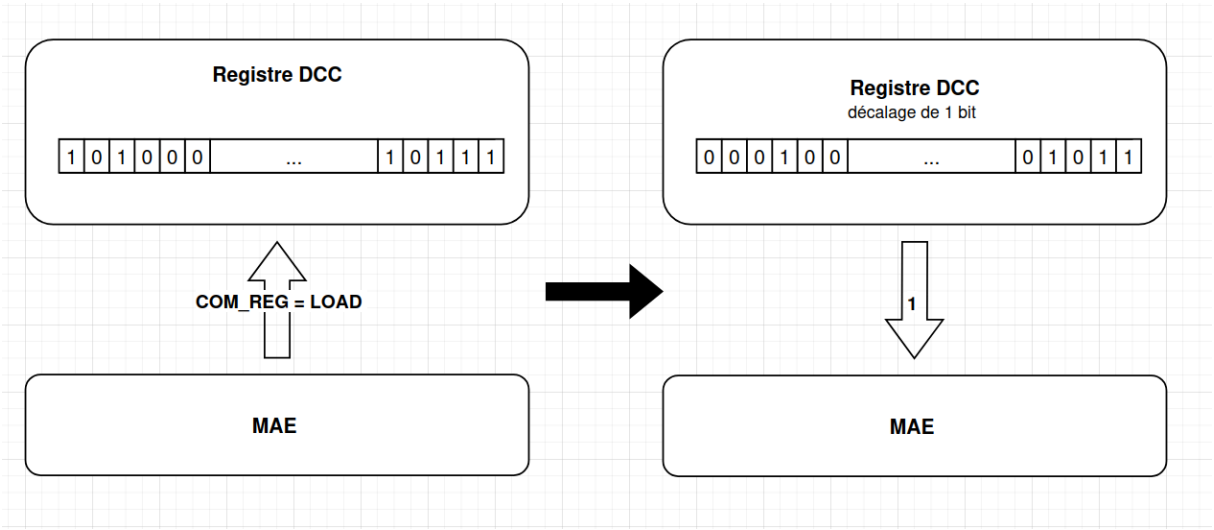


Fig. 4. – Envoie d'un bit de la trame

Machine à états (MAE)

La Machine à États est l'organe de contrôle séquentiel du système. Elle assure la sérialisation de la trame de 51 bits en décidant du cadencement des lectures, des envois et des temps d'attente. Le passage d'un état à un autre s'effectue à chaque coup d'horloge selon des conditions spécifiques.

Ports de la MAE

Les ports de la machine sont définis comme suit :

- **Entrées :** `Reset` (asynchrone),
- `Clk_In` (100MHz),
- `FIN_TEMPO`,
- `FIN_0`,
- `FIN_1`,
- `BIT_LU`.
- **Sorties :** `START_TEMPO`,
- `GO_0`,
- `GO_1`,
- `COM_REG`.

Tous les signaux sont de taille 1 bit, à l'exception de `COM_REG` qui est un vecteur de 2 bits permettant de piloter le registre DCC:

- **00 :** Aucune action (maintien).
- **01 :** Chargement parallèle (Load).
- **10 :** Décalage à droite (Shift).

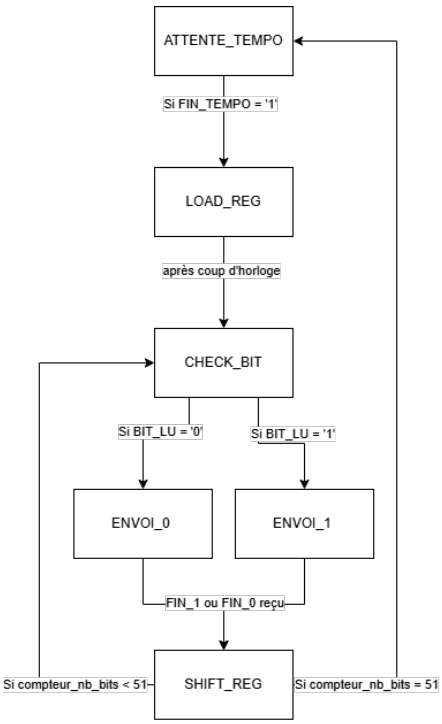


Fig. 5. – Diagramme de la Machine à États (MAE)

Description des États

L'automate repose sur les états principaux suivants :

1. **ATTENTE_TEMPO (Repos) :** Lance un chronomètre de 6 ms via `START_TEMPO` pour respecter la consigne entre deux paquets DCC. L'état est maintenu tant que `FIN_TEMPO` n'est pas reçu.
2. **LOAD_REG (Chargement) :** La MAE ordonne au registre d'ingérer les 51 bits de la trame via la commande `01`.
3. **CHECK_BIT (Aiguillage) :** Analyse le bit de droite (`BIT_LU`). Si c'est un "1", transition vers `ENVOI_1`, sinon vers `ENVOI_0`.
4. **ENVOI_1 / ENVOI_0 :** Active les sous-modules DCC correspondants (`GO_1` ou `GO_0`) et attend le signal d'acquittement (`FIN_1` ou `FIN_0`) indiquant que l'impulsion sur les rails est terminée.
5. **SHIFT_REG (Décalage) :** Demande le décalage du registre (`10`). Si le compteur atteint 51 bits, la trame est finie et on retourne au repos ; sinon, on boucle sur l'envoi du bit suivant

Validation et Simulation

Pour valider le comportement séquentiel de notre machine à états, nous avons essayé de lui donner des entrées comme le reset initialement pour s'assurer qu'il fonctionne bien. Nous avons aussi testé le changement d'état avec le chronomètre de 6 ms sauf que dans notre cas on en a simulé un plus rapide pour vérifier si ça fonctionnait bien :

- **Test de temporisation :** Vérification du passage de l'état de repos à l'activation après simulation du délai.
- **Test d'aiguillage (Bit "1") :** En forçant `BIT_LU` à "1", on observe l'activation de `GO_1`. Après l'envoi de l'acquittement `FIN_1`, la MAE transite correctement vers `SHIFT_REG`.

```
-- Extrait du test de l'état ENVOI_1
BIT_LU <= '1';
wait until GO_1 = '1';
wait for clk_period * 3;
FIN_1 <= '1';
```

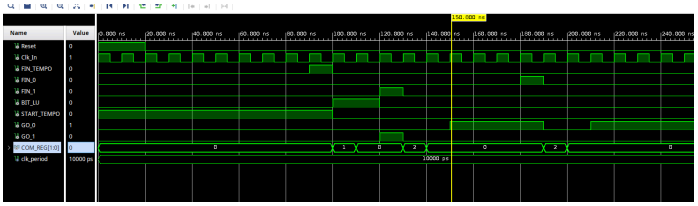


Fig. 6. – Chronogrammes de simulation

Test d'aiguillage (Bit "0") : L'injection d'un "0" logique confirme l'activation de la sortie `GO_0` et l'aiguillage vers l'état `ENVOI_0`.

DCC_Bit 1 et DCC_Bit_0

Ces deux modules ont pour rôle de générer un signal positif puis nul selon les spécifications temporelles du protocole DCC. Ils partagent la même machine à états.

Fonctionnement et interaction :

- Chaque bloc reste au niveau bas ("0") tant qu'il n'est pas sollicité.
- Lorsqu'un bloc reçoit le signal d'activation (`GO_1` ou `GO_0`) de la MAE globale, il génère son impulsion spécifique.
- Une fois la transmission du bit achevée, le bloc émet un signal d'acquittement vers la MAE globale.

Par construction, un seul des deux blocs peut être actif à la fois. Les signaux de sortie de chaque bloc (`DCC_1` et `DCC_0`) sont finalement combinés à travers une porte OU logique pour former le signal unique `SORTIE_DCC` envoyé vers la voie.

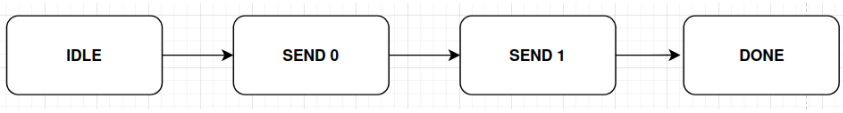


Fig. 7. – Machine à états interne des blocs de génération de bits

Centrale DCC

Le bloc `Centrale_DCC` constitue le cœur de l'architecture matérielle dédiée au contrôle du train. Il encapsule de manière structurelle l'ensemble des composants vus précédemment : la logique de contrôle (la MAE globale), la chaîne de données (le registre DCC à décalage), le bloc de temporisation, ainsi que les générateurs de bits DCC_Bit_1 et DCC_Bit_0.

Ce bloc centralise la sérialisation et la mise en forme du signal de commande avant sa transmission. Dans la suite du projet, cette architecture autonome est directement réutilisée pour être encapsulée au sein d'un composant IP périphérique (AXI Wrapper) connecté au processeur Microblaze.

TOP_DCC

Le module `TOP_DCC` représente l'entité de niveau supérieur (**Top-Level**) de cette première phase de conception matérielle. Il a pour but d'interconnecter le bloc opérationnel `Centrale_DCC` avec le `Générateur de Trames de Test` piloté par les interrupteurs physiques de la carte Basys.

C'est ce composant global qui est synthétisé, implémenté puis injecté sur le FPGA. Il permet de valider le comportement dynamique du matériel de manière autonome — notamment en vérifiant la conformité des trames générées à l'aide d'un oscilloscope — avant de substituer définitivement le générateur de test par le système logiciel embarqué.

Intégration de la Centrale DCC comme IP Microblaze

Dans cette étape, le projet évolue vers un système mixte matériel/logiciel. La préparation des trames est désormais gérée par un programme en C exécuté par le processeur Microblaze, tandis que l'envoi physique du signal reste assuré par notre matériel VHDL.

Architecture globale du système et interconnexion par Bus AXI

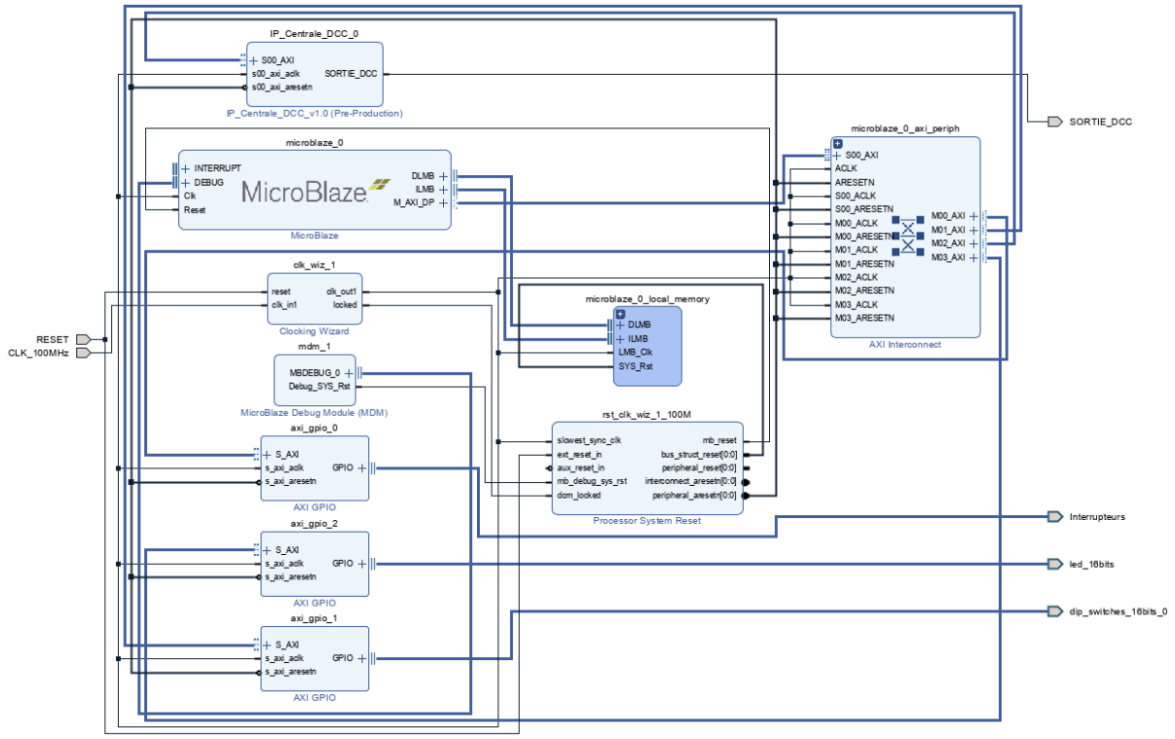


Fig. 8. – Block Design du système Microblaze

Le schéma ci-dessus illustre l'architecture complète. Le cœur du système est le processeur Microblaze_0. Il communique avec l'ensemble de ses périphériques par l'intermédiaire du bus standardisé microblaze_0_axi_periph (`AXI Interconnect`). L'architecture comprend également les blocs nécessaires à son fonctionnement de base : gestion de l'horloge (`clk_wiz`), réinitialisation (`rst_clk_wiz`) et mémoire locale (`local_memory`).

Conception de l'AXI Wrapper et choix des registre d'interface

Afin de connecter notre architecture VHDL au Microblaze, nous l'avons transformée en un composant IP sur mesure (`IP_Centrale_DCC_0`). Un composant intermédiaire, l'AXI Wrapper, a été généré pour assurer l'interface. Le processeur Microblaze vient écrire les données de la trame DCC à envoyer dans les registres (slv_reg) de cet AXI Wrapper. Notre composant matériel récupère ensuite ces données pour générer le signal final sur le port de sortie `SORTIE_DCC`.

Gestion des Entrées/Sorties par le Microblaze (GPIO Interrupteurs et Boutons)

La lecture des commandes physiques de la carte Basys (boutons, interrupteurs) n'est plus reliée directement à la centrale DCC. Nous avons intégré des contrôleurs `AXI GPIO` :

- **axi_gpio_0 / axi_gpio_1 :** Dédiés à la lecture des interrupteurs (`Interrupteurs` et `dip_switches`), permettant au code C de décoder la commande souhaitée par l'utilisateur.
- **axi_gpio_2 :** Dédié au pilotage des LEDs (`led_16bits`) pour rendre le debug sur l'état du système plus facile.

Ainsi, le code logiciel interroge périodiquement ces GPIOs. Lorsqu'une action est détectée, il prépare la trame DCC correspondante et l'envoie à l'IP matérielle via le bus AXI.

Architecture Logicielle

Organigramme de constitution des trames DCC

Le système s'utilise via les 8 interrupteurs physiques (switchs). Leur nombre est limité donc on a dû penser à une architecture logicielle flexible. Plutôt que d'assigner un switch à une seule fonction, les interrupteurs ont des rôles dynamiques en fonction du « mode » sélectionné, à l'exception du premier switch qui conserve le même rôle : le choix de l'adresse.

Configuration Globale des Interrupteurs
L'attribution générale des 8 switchs (`Switch(0)` à `Switch(7)`) est structurée ainsi :

Interrupteur(s)	Fonctionnalité / Rôle logiciel
`Switch(0)`	Sélection dynamique de l'adresse du train (Adresse n°4 ou n°5) pour éviter l'encodage en dur.
`Switch(1)` `- Switch(2)`	Sélection du mode de fonctionnement (Modes 0, 1, 2 ou 3 en binaire).
`Switch(3)` `- Switch(6)`	Arguments des fonctions (Vitesse du train ou activation des fonctions F0 à F4).
`Switch(7)`	Argument spécial dépendant du mode choisi (Direction ou sélection de sous-groupe).

Description des Modes
les modes concernent les 4 groupes de fonction données dans le sujet :
- Le mode 00 concerne les fonction de marche avant/arrière.
- Le mode 01 concerne les fonctions F0 à F4.
- Le mode 10 concerne les fonctions F5 à F12.
- Le mode 11 concerne les fonctions F13 à F20 plus complexes.

Gestion de l'Adresse (`Switch(0)`)

Afin de permettre la cohabitation de plusieurs modules ou de s'adapter au circuit, le `Switch(0)` permet de basculer l'adresse de destination de la trame DCC (par exemple entre l'adresse standard 4 et l'adresse alternative 5). Cela évite de devoir reprogrammer la carte FPGA en cas de conflit d'adresse sur les rails.

Sélection du Mode (`Switch(1)` et `Switch(2)`)

Ces deux interrupteurs forment un vecteur binaire de 2 bits qui définit 4 états de fonctionnement (de `00` à `11`). Ce mode change la manière dont le générateur de trames va interpréter les switchs restants.

Rôle des Switchs d'Arguments (`Switch(3)` à `Switch(6)`)
Le comportement de ces interrupteurs s'adapte selon le mode actif :
- **En Mode 0 (Contrôle du mouvement) :** Ils codent numériquement la consigne de vitesse du train, de `00000` (arrêt complet) à `11111` (vitesse maximale, correspondant au Step 28).

- **Dans les autres modes (Fonctions annexes) :** Chaque bit agit comme un interrupteur individuel de type Tout-ou-Rien (*On/Off*) pour activer ou désactiver une fonction spécifique (par exemple, le bit 4 contrôle la fonction F0 pour l'allumage des phares).

Rôle du Switch Spécial (`Switch(7)`)

Cet interrupteur remplit une fonction critique et variable selon la configuration :

- **En Mode 0 :** Il définit le bit de direction (D) dans l'octet de commande (`01DXXXXX`). Un état bas (`0`) configure une marche arrière, tandis qu'un état haut (`1`) ordonne une marche avant.
- **En Mode 2 (Fonctions F5 à F12) :** Il sert de bit de sélection de sous-groupe (S) dans l'octet de commande (`101SXXXX`). Il permet de commuter les switchs d'arguments pour piloter soit le groupe de fonctions F5-F8 (lorsque $S = 1$), soit le groupe F9-F12 (lorsque $S = 0$).
- **Pour les modes restants :** Il est rétrogradé au rang d'argument simple pour activer ou désactiver une option supplémentaire

Interface Matériel/Logiciel (IP Custom et Application Vitis)

Pour assurer la communication entre le processeur (Software) et les modules de génération DCC (Hardware), le développement d'un bloc d'IP sur mesure est indispensable. Ça nous permet au code C de piloter directement le comportement physique du système.

Architecture de l'IP Custom (Côté Hardware)

Le processeur communique avec les périphériques via un bus de données standard codé sur 32 bits. Cependant, la trame DCC fait 51 bits, ce qui dépasse la capacité d'un seul registre du bus.

Pour contourner cette limitation, l'architecture de l'IP intègre deux registres esclaves de 32 bits chacun (`slv_reg0` et `slv_reg1`), qui nous donne un espace adressable de 64 bits :

- **`slv_reg0` :** Reçoit les 32 bits de poids faibles de la trame.
- **`slv_reg1` :** Reçoit les 19 bits de poids forts restants.

Dans la zone dédiée aux modifications utilisateur de l'IP `-- Add user logic here`, la trame globale de 51 bits est reconstruite instantanément par concaténation avant d'être injectée directement dans l'entrée `TRAME_IN` de la centrale :

```
-- Reconstruction de la trame DCC au sein de l'IP custom
trame_51_bits <= slv_reg1(18 downto 0) & slv_reg0;
```

C'est également dans cette section de l'IP que sont interconnectés les signaux d'entrée/sortie de la machine à états ainsi que les compteurs de temporisation matériels.

Développement de l'Application (Côté Software sous Vitis)

La logique de l'application est dans le fichier main.c. Tout se fait dans une boucle infinie (while(1)) qui exécute les étapes suivantes :

Initialisation et Lecture des E/S : Configuration des entrées logiques (Gpio_Switches, Gpio_Buttons) et des sorties visuelles (Gpio_Leds). La lecture de l'état des commutateurs physiques est réalisée via la fonction de l'API : XGpio_DiscreteRead().

Génération Dynamique de la Trame : Lorsqu'une action sur un bouton poussoir est détectée, le programme assemble la trame. Les variables de type uint8_t stockent les champs de commande intermédiaires, tandis que des variables uint32_t reçoivent les résultats finaux.

Opérations Binaires et Checksum : Des opérations de masquage binaire (&) et de décalage de bits (>>) permettent d'isoler dynamiquement l'adresse cible (4 ou 5), le mode actif et les arguments associés. Le champ de contrôle (Checksum) est calculé par une suite d'opérations logiques XOR (^).

Écriture sur le Bus : La trame finale de 51 bits est coupée en deux paquets de 32 bits (reg0 et reg1), puis envoyée sur le bus AXI vers les registres esclaves de l'IP matérielle.

Anti-rebond Logiciel : En fin de cycle, une boucle d'attente bloquante maintient le programme tant que le bouton poussoir reste enfoncé, sécurisant ainsi l'envoi d'une trame unique par impulsion et évitant les rebonds parasites.

```
code C
while(1) {
sw_values = XGpio_DiscreteRead(&Gpio_Switches, 1);

if (button_pressed) {
checksum = adresse ^ commande_octet1 ^ commande_octet2;

reg0 = (uint32_t)(trame_complete & 0xFFFFFFFF);
reg1 = (uint32_t)((trame_complete >> 32) & 0x7FFF);

Xil_Out32(IP_BASE_ADDRESS, reg0);
Xil_Out32(IP_BASE_ADDRESS + 4, reg1);

while(XGpio_DiscreteRead(&Gpio_Buttons, 1) == button_pressed);
}
}
```

pourquoi Xil_Out32 ? On a cherché à droite à gauche et on a trouvé cette fonction fournie directement par Xilinx qui sert à écrire une donnée de 32 bits directement à une adresse mémoire bien précise ça nous a semblé pertinent et utile pour notre projet alors on l'avons utilisée pour écrire dans les slaves registers.

Bilan d'avancement :
Notre projet est presque complet ; cependant, un comportement anormal fait que nous ne recevons que 32 bits au lieu des 51 attendus. La trame précédente est écrasée par la suivante sans être traitée par le matériel. C'est la dernière correction que nous aurions dû apporter pour garantir le bon fonctionnement de notre projet.

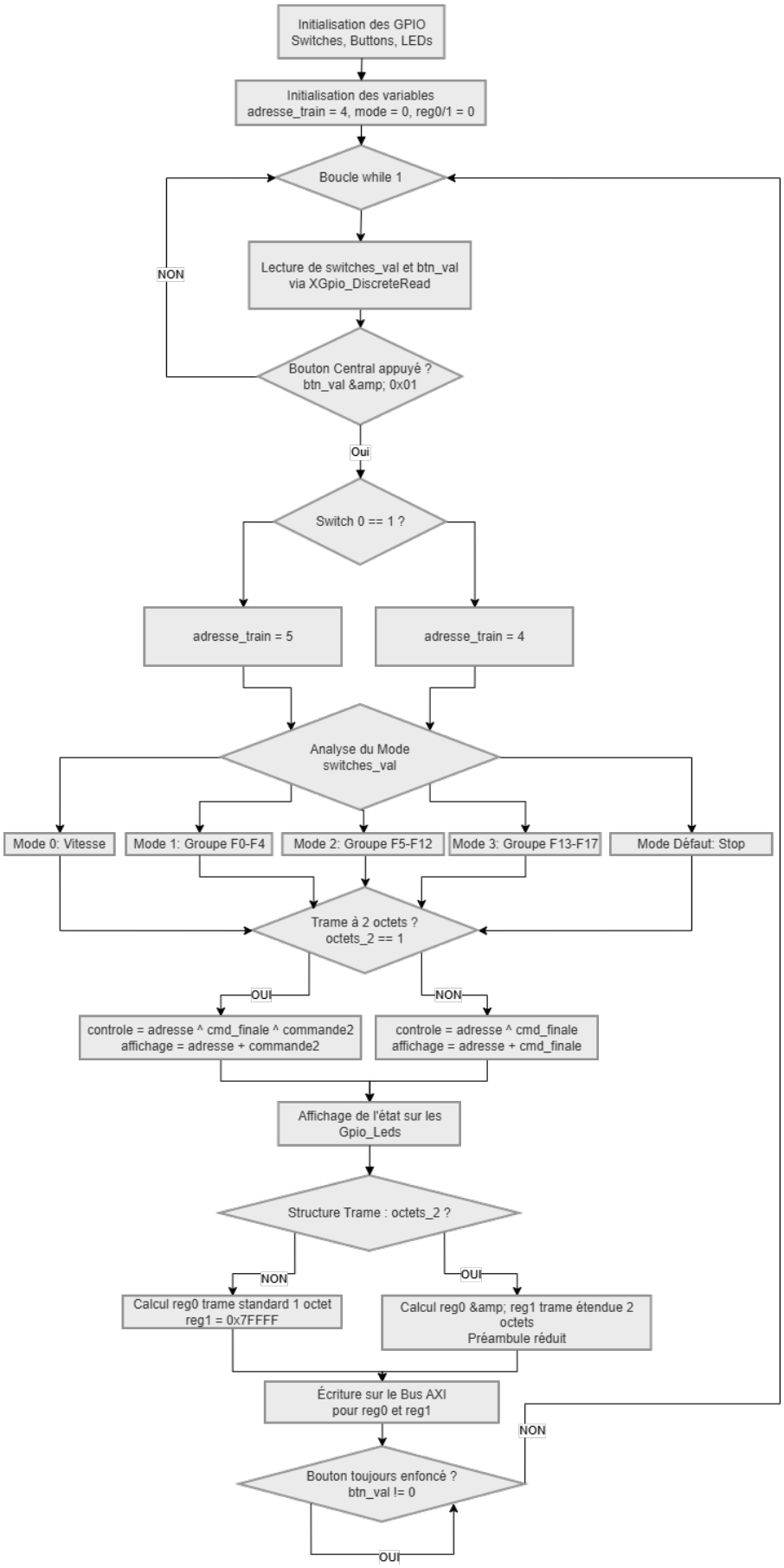


Fig. 9. – Organigramme de notre code C